

Chamourai
Rapport de soutenance

Les Chatpitres



Table des matières

1	Introduction	4
2	Répartition des rôles	5
3	Parcours de jeu et fonctionnalités	6
3.1	Une progression guidée	6
3.2	Exploration et carte	6
3.3	Narration environnementale	8
4	Conception technique des cartes	9
4.1	Cartes produites avec <i>Tiled</i>	9
4.2	Collisions des murs et de la grille	9
4.3	Pourquoi ce choix est durable	10
5	Système de combat	11
5.1	Épée orientée par la souris	11
5.2	Esquive directionnelle	12
6	Ennemis et adversaires majeurs	13
6.1	Squelettes et nécromanciens	13
6.2	Recherche de chemin	13
6.3	Adversaires intermédiaires et clés	14
6.4	Le golem, adversaire final	15
7	Interface, graphismes et son	17
7.1	Direction graphique et animations	17
7.2	Retours des personnes ayant essayé le jeu	17
7.3	Menus et informations en jeu	17
7.4	Musiques et effets sonores	19
7.5	Installation et désinstallation	19
8	Mode à deux joueurs en coopération	21
8.1	Principe de connexion	21
8.2	Progression et équilibrage à deux	21
8.3	Mort, observation et reprise	21
9	Difficultés rencontrées et choix de conception	23
9.1	Faire évoluer une carte sans réécrire le jeu	23
9.2	Concilier lisibilité et ambition des adversaires majeurs	23
9.3	Synchroniser des systèmes devenus interdépendants	23
9.4	Intégrer le travail de plusieurs branches	24

10 Communication et organisation	25
10.1 Outils de communication	25
10.2 Gestion des versions	25
10.3 Suivi des tâches	26
11 Bilan et perspectives	27
12 Annexes	28

1 Introduction

Chamouraï est un jeu d'action et d'aventure en vue du dessus, réalisé en *Python* avec *Pygame*. Le joueur incarne un chat samouraï qui doit progresser d'une forêt vers un donjon, affronter des ennemis de plus en plus dangereux, réunir deux clés puis vaincre un golem. Notre objectif était de proposer une expérience immédiatement jouable, mais qui demande de maîtriser progressivement le placement, l'esquive et le rythme des attaques.

Le projet a beaucoup évolué depuis sa première version. Le prototype initial reposait principalement sur le déplacement du joueur, l'épée orientée à la souris, quelques ennemis et une carte. La version actuelle comprend un apprentissage guidé, des objectifs successifs, plusieurs familles d'ennemis, des adversaires majeurs, un verrouillage du donjon par clés, une carte consultable, des musiques contextuelles et un mode à deux joueurs avec gestion de la mort et de la reprise de partie.

Le jeu repose sur un principe d'apprentissage par l'échec : perdre fait partie de l'apprentissage. Nous avons cependant cherché à rendre les échecs compréhensibles. Les animations annoncent les attaques, la barre de vie de l'adversaire majeur donne un repère clair et la reprise depuis le début du donjon évite de répéter inutilement l'introduction.

L'identité de *Chamouraï* vient du rapprochement de deux univers que nous apprécions : l'ambiance de samouraïs et les paysages japonais de *Ghost of Tsushima*, associés au personnage félin de *Stray*. Cette association nous a conduit à imaginer un héros à la fois attachant et combattant : un chat samouraï placé dans un monde menacé, mais encore riche en couleurs et en détails.



FIGURE 1 – Vue générale d'une partie de *Chamouraï*

2 Répartition des rôles

Membre	Responsabilités principales
<i>Baptiste LOISEAU</i>	Chef de projet, conception des cartes, jouabilité, intégration graphique et équilibrage des combats
<i>Lucas HUET</i>	Interfaces, menus, musiques, effets sonores et réglages du son
<i>Loan BLANPAIN</i>	Site internet, scénario, univers et communication du projet
<i>Simon BOURGEOIS</i>	Intelligence artificielle, déplacements des ennemis et multijoueur

Cette répartition nous a donné des domaines de responsabilité clairs, mais le jeu ne peut pas être assemblé en isolant complètement les tâches. Par exemple, une nouvelle zone créée dans *Tiled* influence les collisions, l'intelligence artificielle, les objectifs, les sons et la synchronisation réseau. Une partie importante du travail a donc été l'intégration : vérifier ensemble les ajouts de chacun et résoudre les conflits lorsque plusieurs fonctionnalités modifiaient la boucle principale du jeu.

3 Parcours de jeu et fonctionnalités

3.1 Une progression guidée

Au lancement d'une partie, le joueur arrive dans la zone extérieure. Plutôt que de lui demander de découvrir toutes les commandes au milieu d'un combat difficile, nous avons ajouté une suite d'objectifs affichée en haut à gauche de l'écran :

1. parler au vieil homme afin de découvrir le parchemin et l'interaction ;
2. frapper trois fois afin de comprendre l'utilisation de l'épée ;
3. esquiver trois fois pour apprendre la mobilité défensive ;
4. éliminer trois squelettes dans la forêt ;
5. entrer dans le donjon ;
6. ouvrir la carte avec la touche M ;
7. trouver les deux clés permettant d'ouvrir la grille.

Le portail vers le donjon n'est accessible que lorsque les premiers objectifs sont validés. Ce choix évite qu'un joueur se retrouve face aux adversaires majeurs sans avoir utilisé les actions essentielles. À deux joueurs, les éliminations utiles à l'apprentissage sont synchronisées : les deux joueurs partagent ainsi la même progression au lieu de se retrouver bloqués à des étapes différentes.



FIGURE 2 – Premier objectif de l'apprentissage guidé

3.2 Exploration et carte

Dans le niveau principal, la touche M affiche une carte simplifiée. Elle permet au joueur de localiser sa position dans un environnement plus vaste, sans supprimer la nécessité d'explorer. La carte est volontairement fonctionnelle : elle montre les zones utiles et le joueur, mais ne remplace pas le déplacement réel dans le niveau.

La grille qui ferme l'accès au golem constitue l'objectif central du donjon. Deux adversaires majeurs intermédiaires laissent tomber chacun une clé lorsqu'ils sont vaincus.

Dans la zone devant la grille, un message indique le nombre de clés requis ; après collecte des deux clés, le joueur peut appuyer sur E pour ouvrir le passage. Cette interaction relie combat et exploration : éliminer les adversaires majeurs a une conséquence visible sur la carte.



FIGURE 3 – Carte du niveau et position du joueur

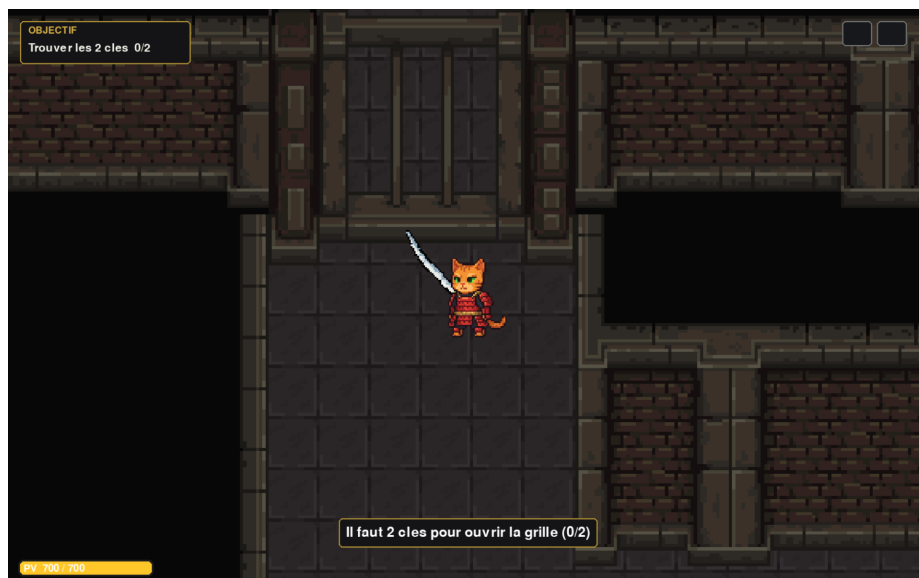


FIGURE 4 – Grille fermée et emplacements de clés

3.3 Narration environnementale

Un vieil homme animé est présent dans la zone de départ. Lorsque le joueur s'approche, une icône animée de touche E apparaît ; l'interaction ouvre un parchemin en grand à l'écran. Pendant sa lecture, le déplacement, l'esquive et l'attaque sont bloqués afin de ne pas provoquer une action involontaire derrière l'interface.

Le parchemin a deux rôles. Il présente d'abord les commandes principales au joueur, puis il introduit la situation : la population est terrorisée par les monstres du donjon, qui commencent désormais à rôder jusque dans la forêt. Cette scène donne ainsi un objectif au personnage avant son premier affrontement, sans imposer une longue séquence d'explication séparée du jeu. Nous avons ralenti l'animation de la touche d'interaction afin qu'elle attire l'attention sans clignoter de manière gênante.

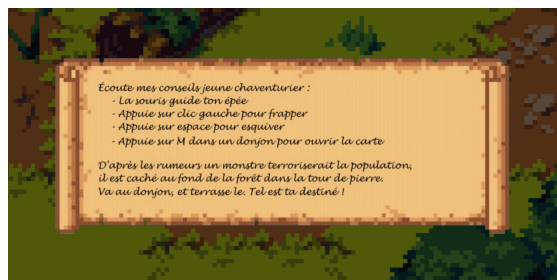


FIGURE 5 – Interaction avec le vieil homme et lecture du parchemin

4 Conception technique des cartes

4.1 Cartes produites avec *Tiled*

Les niveaux sont dessinés avec *Tiled*, qui permet de superposer des calques de tuiles et des calques d'objets. Le programme charge les fichiers `.tmx` et convertit leur contenu en éléments de jeu. Cette méthode est beaucoup plus souple que d'écrire les positions directement en *Python* : la carte peut être retouchée visuellement sans modifier la logique centrale.

La forêt devait être davantage qu'une simple zone de départ. Nous souhaitions créer un environnement immersif et particulièrement détaillé, afin que le joueur ressente la différence entre un extérieur encore vivant, mais désormais dangereux, et l'intérieur minéral du donjon. Le travail par calques dans *Tiled* nous a permis d'ajouter du relief visuel et des éléments de décor sans perdre la lisibilité des chemins et des combats.

Pour les ennemis, chaque objet *Tiled* portant le nom `Skeleton`, `Necromancer`, `SkeletonBoss`, `NecromancerBoss` ou `Golem` crée automatiquement le monstre correspondant. Ajouter un squelette revient donc à déposer un nouvel objet sur la carte. Le même principe est utilisé pour les zones d'apparition et d'interaction.

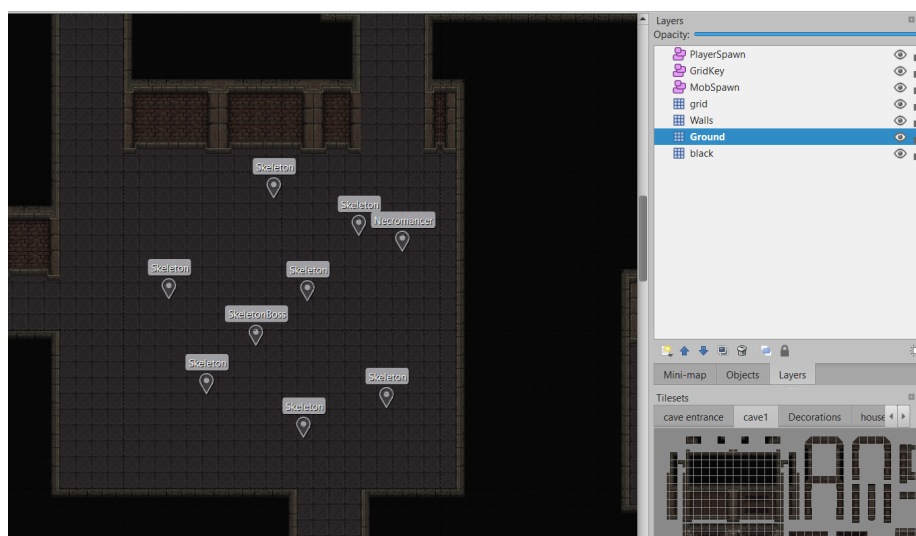


FIGURE 6 – Édition du niveau dans *Tiled*

4.2 Collisions des murs et de la grille

La première carte utilisait des collisions définies objet par objet, ce qui devenait long et source d'erreurs dès que le décor changeait. Nous avons choisi de rendre bloquante toute tuile non vide du calque `Walls`. Le responsable de la conception des niveaux peut ainsi dessiner un mur et obtenir immédiatement une collision cohérente.

L'accès à l'adversaire final a nécessité une distinction importante. Le calque de tuiles `Grid` contient l'apparence et les collisions de la grille, tandis que le calque d'objets `GridKey` définit uniquement la zone où le message d'interaction apparaît. Confondre ces deux types

de calques entraînait une erreur au chargement, car une tuile fournit des coordonnées et un identifiant tandis qu'un objet *Tiled* fournit un rectangle. Après ouverture, le calque visuel de la grille est masqué et ses collisions sont retirées simultanément.

4.3 Pourquoi ce choix est durable

La génération pilotée par la carte réduit les dépendances entre la programmation et la création des niveaux. Elle impose cependant une convention : les noms des calques et des objets doivent être exacts. Nous avons accepté cette contrainte, car elle est facile à documenter et bien plus pratique que d'ajouter une ligne de code pour chaque nouvel ennemi ou obstacle.

5 Système de combat

5.1 Épée orientée par la souris

Le combat repose sur une coordination clavier-souris. Le joueur se déplace au clavier et indique avec le curseur l'endroit vers lequel il souhaite orienter son épée. Pour obtenir cette orientation, nous considérons le segment qui relie le personnage au curseur. Ce segment forme un angle avec l'axe horizontal de l'écran.

La méthode utilisée est trigonométrique : à partir de l'écart horizontal et de l'écart vertical entre le joueur et la souris, une fonction d'arc tangente permet de retrouver l'angle de visée. Cette méthode a l'avantage de fonctionner dans toutes les directions, y compris en diagonale, contrairement à un système limité à « gauche » et « droite ». L'épée peut ainsi suivre naturellement le curseur autour du personnage.

Une difficulté est apparue lorsque le joueur changeait très rapidement de côté pendant une attaque : l'animation pouvait sembler recommencer. Nous avons donc choisi de conserver l'orientation prise au début d'un coup jusqu'à sa fin. Pour le joueur, le résultat est plus clair : chaque action déclenche une seule animation complète.

Cette mécanique a constitué l'une des parties les plus difficiles du projet. À la différence d'une attaque dirigée dans seulement quatre sens, elle demande de lier en continu le mouvement de la souris à l'orientation de l'arme. Elle nous a notamment obligés à mobiliser des notions de géométrie et de trigonométrie pour obtenir un comportement fluide et compréhensible.



FIGURE 7 – Attaque orientée par le curseur dans deux directions

5.2 Esquive directionnelle

L'esquive fonctionnait initialement seulement sur un axe horizontal : regarder à droite envoyait le joueur vers la gauche et inversement. Ce comportement était insuffisant pour les combats en arène. L'esquive utilise désormais le vecteur opposé à la souris : un curseur placé en bas à droite provoque une esquive vers le haut à gauche.

Pendant l'esquive, les déplacements normaux sont bloqués. Sans ce verrouillage, le joueur pouvait modifier artificiellement sa trajectoire et rendre l'action difficile à anticiper. Cette décision améliore aussi l'équilibrage : l'esquive est une action forte, mais elle engage le joueur pendant une courte durée.

6 Ennemis et adversaires majeurs

6.1 Squelettes et nécromanciens

Le squelette constitue l'ennemi rapproché de base : il poursuit le joueur et attaque à courte portée. Le nécromancien introduit une menace à distance et oblige à se déplacer plutôt qu'à attendre derrière son épée. Chacun existe également en version renforcée, avec une apparence agrandie, une barre de vie visible au-dessus de sa tête, davantage de vie, une vitesse supérieure et des attaques plus fréquentes.

Lorsque plusieurs squelettes poursuivaient le joueur, ils avaient tendance à se superposer. Un seul coup pouvait alors toucher tout le groupe sans réel risque. Nous avons ajouté des zones de collision entre ennemis et un espacement dans leur recherche de trajectoire. L'épée peut toujours toucher plusieurs ennemis si son arc les atteint, ce qui récompense un bon placement, mais le groupe n'arrive plus comme une masse immobile.

6.2 Recherche de chemin

Les monstres utilisent une recherche de chemin basée sur A^* . Nous avons préféré A^* à *Dijkstra* car notre objectif n'est pas d'explorer toutes les distances possibles, mais de diriger efficacement un ennemi vers une cible connue en tenant compte des murs. Son heuristique limite la quantité de cases inspectées et convient à une carte découpée en tuiles.

L'intégration a été délicate : un ennemi doit éviter les murs de *Tiled*, garder une distance raisonnable avec les autres ennemis et continuer à attaquer lorsque le joueur est accessible. Un trajet mathématiquement court n'est pas toujours agréable en jeu, notamment lorsque tous les ennemis choisissent exactement le même passage. L'espacement ajouté aux squelettes répond donc à un besoin de jouabilité autant qu'à un problème de collision.



FIGURE 8 – Affrontement avec différents ennemis

6.3 Adversaires intermédiaires et clés

Le *SkeletonBoss* et le *NecromancerBoss* ne sont pas seulement des ennemis agrandis : leurs statistiques renforcées donnent deux combats intermédiaires qui préparent à l'adversaire final. À leur mort, ils déposent chacun une clé représentée par un objet ramassable. Ce choix rend la progression immédiatement lisible : le joueur voit l'objet obtenu et le compteur de clés confirme son avancement.



FIGURE 9 – Adversaire intermédiaire et clé déposée

6.4 Le golem, adversaire final

Le golem est le point culminant du niveau. Son échelle est volontairement imposante et sa barre de vie est affichée en haut de l'écran dès que la grille tombe. En première phase, il dispose d'une attaque de mêlée et d'un tir de bras. À partir de la moitié de sa vie, son apparence devient lumineuse, ses statistiques augmentent et il obtient une attaque supplémentaire : une onde de choc bleue en arc de cercle.

Le choix du golem est aussi issu d'une contrainte de production. Nous avons décidé d'utiliser des ressources graphiques gratuites ; parmi celles qui correspondaient à l'esthétique du jeu, le golem était le seul adversaire disposant d'un ensemble d'attaques assez complet et varié pour construire un véritable combat final. Cette contrainte est devenue une opportunité : sa silhouette imposante correspond bien à une menace enfermée au fond du donjon.

Nous avons envisagé un rayon lumineux continu sortant de sa tête. Sa gestion s'est révélée difficile à rendre fiable et lisible : placement exact de l'origine, feuille d'animation disposée verticalement, collision d'un rayon long et synchronisation des images d'animation. Nous avons remplacé cette idée par une onde de choc. Elle conserve le rôle recherché, c'est-à-dire forcer le joueur à esquiver en seconde phase, mais sa forme, sa vitesse et sa zone de contact sont plus simples à comprendre et à régler.

La mort du golem joue une animation plus longue et laisse son corps visible, ce qui donne du poids à la victoire. L'écran de fin propose ensuite un retour au menu principal.

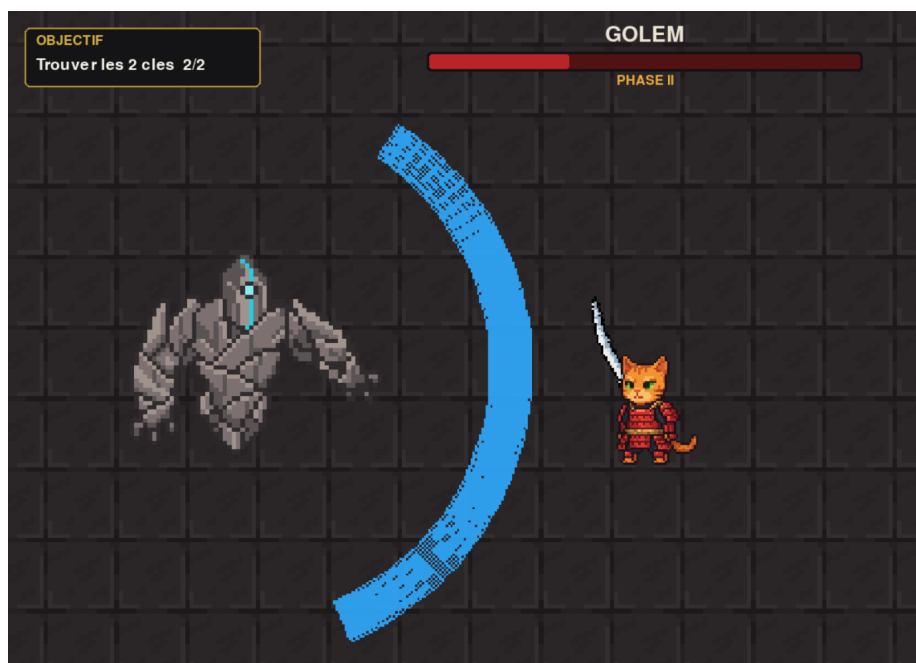


FIGURE 10 – Combat contre le golem en seconde phase

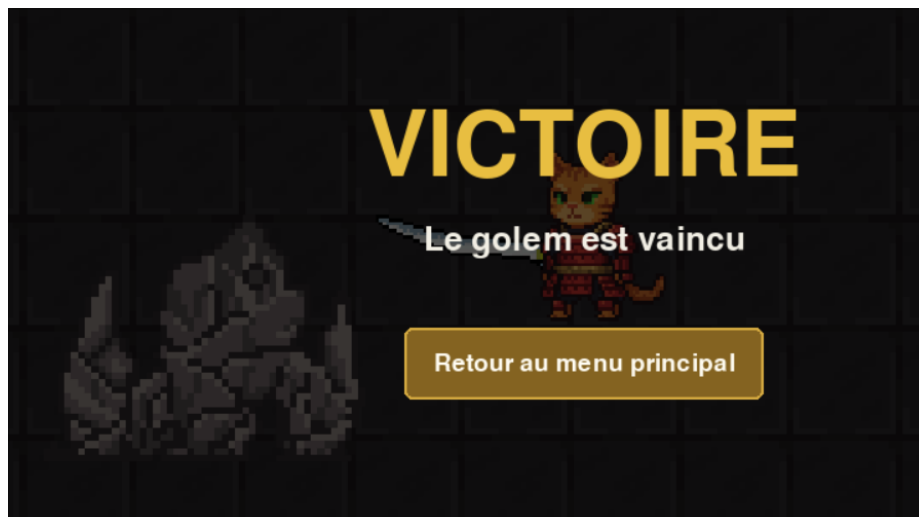


FIGURE 11 – Écran de victoire après la défaite du golem

7 Interface, graphismes et son

7.1 Direction graphique et animations

Le jeu adopte un style de dessin en pixels en vue du dessus. Le personnage principal a été généré puis retravaillé pour correspondre à l'univers ; ses animations ont été adaptées aux actions principales. Les feuilles de tuiles et plusieurs monstres ont été sélectionnés dans des ressources compatibles avec notre direction artistique, puis intégrés à nos contraintes d'échelle et d'animation.



FIGURE 12 – Feuille d'animation de marche du joueur

L'ajout du vieil homme, des clés et des différents adversaires majeurs a demandé une attention particulière aux dimensions des images : un adversaire majeur doit sembler plus massif tout en conservant des collisions jouables. Le golem, notamment, utilise des images plus grandes que les autres personnages et des images séparées pour ses projectiles.

7.2 Retours des personnes ayant essayé le jeu

Les personnes qui ont essayé *Chamouraï* ont particulièrement apprécié l'association entre le chat et l'univers samouraï, qui donne rapidement une identité au projet. Le système de combat guidé par la souris a également été perçu comme original. Ces retours ont confirmé l'intérêt de conserver une attaque libre et directionnelle, même si sa réalisation était plus complexe qu'un système de combat orienté uniquement dans quelques directions fixes.

7.3 Menus et informations en jeu

Le menu principal permet d'accéder au jeu à un joueur, au jeu à deux, aux options et à la sortie. Le menu du jeu à deux offre la création ou la connexion à une partie et comprend un bouton de retour, indispensable pour ne pas enfermer le joueur dans un parcours de connexion. En jeu, l'interface affiche les informations utiles au bon moment : objectifs pendant l'apprentissage, clés dans le donjon, carte sur demande et barre du golem lors du combat final.

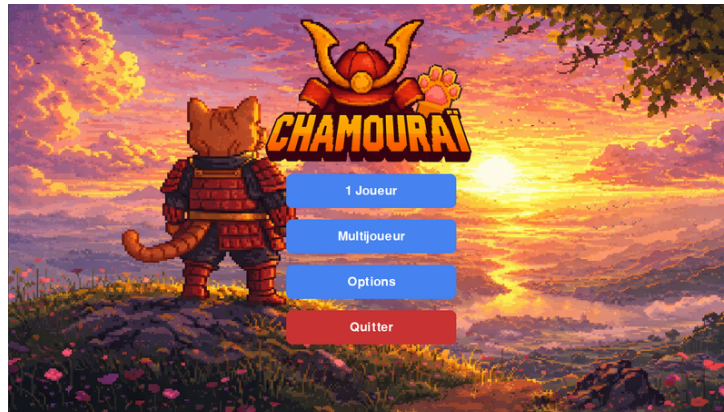


FIGURE 13 – Interface principale du jeu

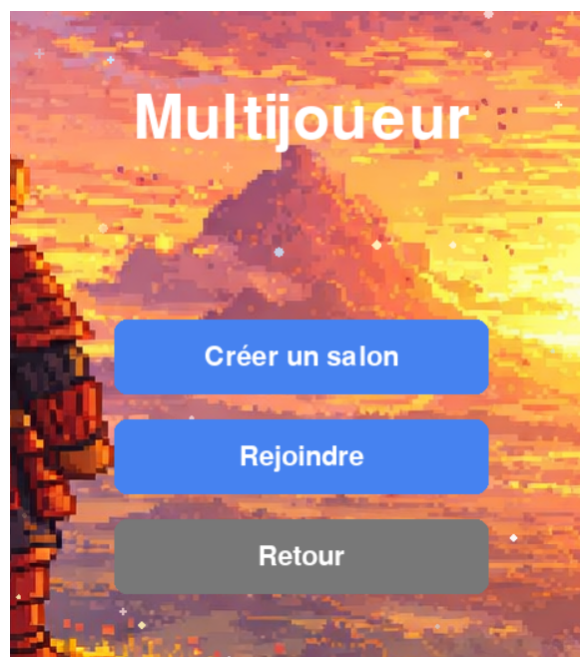


FIGURE 14 – Menu du jeu à deux

7.4 Musiques et effets sonores

Les sons et musiques renforcent la compréhension de l'action : bruit d'attaque, ambiances et transitions musicales accompagnent les différents états de partie. La musique du donjon est lancée lors de l'entrée dans le niveau ou d'une reprise, puis une musique de combat final prend le relais lorsque la grille s'ouvre. Le menu des sons permet au joueur d'ajuster librement le volume, avec un réglage indépendant pour la musique et pour les effets sonores. Il peut ainsi conserver l'ambiance musicale tout en atténuant les bruits d'action, ou au contraire mieux entendre les indications liées au combat.

Nous avons également différencié les sons selon le lieu : les pas sur l'herbe dans la zone extérieure ne sont pas ceux des pas sur la pierre du donjon. Les musiques suivent la même progression dramatique : une ambiance calme accompagne la forêt, une musique plus dynamique souligne l'entrée dans le donjon, puis une musique épique marque l'affrontement contre le golem.

L'intégration sonore a posé un problème fréquent dans un projet en équipe : la boucle principale était également modifiée par les objectifs, les adversaires majeurs et le jeu à deux. Lors de la fusion de ces travaux, il fallait préserver les appels sonores sans perdre les règles de progression ou de reprise. Nous avons donc rattaché la musique à des événements identifiables, comme une transition de carte ou l'ouverture de la grille, plutôt que de déclencher un son de manière diffuse à chaque rafraîchissement d'écran.

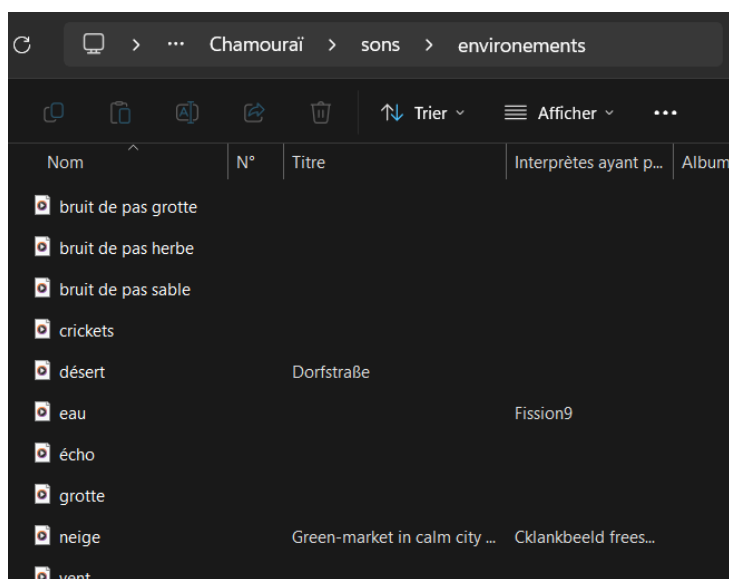


FIGURE 15 – Organisation des ressources sonores

7.5 Installation et désinstallation

Pour rendre *Chamourai* facilement accessible, nous avons préparé un programme d'installation au format exécutable. Son utilisation est volontairement simple : le joueur le lance, choisit l'emplacement où il souhaite installer le jeu, puis peut accéder à celui-ci sans avoir à configurer manuellement son environnement. Un programme de désinstallation est également fourni afin de retirer proprement le jeu de l'ordinateur lorsque l'utilisateur ne souhaite plus le conserver.

Lors de l'exécution du programme d'installation, *Windows* peut néanmoins afficher un avertissement de sécurité. Cet avertissement ne provient pas d'un problème identifié dans le jeu, mais du fait que notre exécutable n'est pas signé par un certificat reconnu par *Microsoft*. En tant qu'équipe étudiante, nous ne sommes pas enregistrés comme éditeur de confiance auprès du système. Pour une diffusion plus large, une évolution importante consisterait donc à signer numériquement le programme d'installation, afin d'identifier clairement son origine et de rassurer davantage les utilisateurs au moment de son lancement.

8 Mode à deux joueurs en coopération

8.1 Principe de connexion

Le mode à deux joueurs est conçu pour la coopération. L'un héberge la partie et l'autre rejoint la session ; l'hôte exécute la logique serveur nécessaire à la cohérence de la partie. Les joueurs envoient leurs actions et leurs états, tandis que les éléments qui doivent être identiques pour tous, notamment les monstres, leurs points de vie, les clés et l'ouverture de la grille, sont synchronisés depuis le serveur.

Ce choix est adapté à un projet de petite taille : il évite une infrastructure en ligne permanente tout en offrant une expérience en coopération. Il demande cependant de ne pas faire confiance uniquement à l'affichage local. Si chaque ordinateur déplaçait ses propres monstres, un adversaire majeur pourrait être mort sur un écran et vivant sur l'autre.

Nous avons volontairement limité ce mode à deux participants. Un mode pour quatre joueurs aurait aussi exigé de prendre en charge les parties à trois et d'équilibrer chaque configuration séparément. Nos cartes sont conçues pour un ou deux personnages : au-delà, l'espace de combat deviendrait encombré et les animations se superposeraient, ce qui rendrait l'action plus difficile à lire. Enfin, l'ordinateur de l'hôte gère le serveur et les échanges de la partie ; multiplier les participants aurait accru sa charge de traitement et la complexité de la synchronisation.

8.2 Progression et équilibrage à deux

La téléportation de la zone extérieure vers le donjon exige désormais que les deux joueurs soient présents dans la zone. Lorsqu'un seul joueur attend, un message l'indique clairement. Cette règle empêche un joueur de changer brutalement la carte de son partenaire pendant qu'il termine un combat ou lit un parchemin.

À deux, tous les monstres possèdent deux fois plus de vie. Sans cette adaptation, les deux épées réduiraient fortement la durée des affrontements, en particulier contre les adversaires majeurs. Le doublement des points de vie est un réglage simple, cohérent et compréhensible pour maintenir l'intérêt des combats en coopération.

8.3 Mort, observation et reprise

Lorsqu'un joueur meurt dans une partie à deux, il disparaît du combat et peut observer la partie : sa caméra suit le partenaire encore vivant. Il peut aussi retourner au menu. Si les deux joueurs meurent, un écran de défaite apparaît. Chacun peut voter pour recommencer ; la partie ne redémarre au début du donjon que lorsque les deux votes ont été reçus.

Cette fonction a nécessité de synchroniser davantage qu'une simple position : état vivant ou mort, vote, remise à zéro des ennemis, des clés et de la grille, point de réapparition et musique. Nous avons retenu un redémarrage au début du donjon plutôt qu'au

tout début du jeu afin de respecter le principe d'apprentissage par l'échec sans imposer à nouveau l'apprentissage guidé.

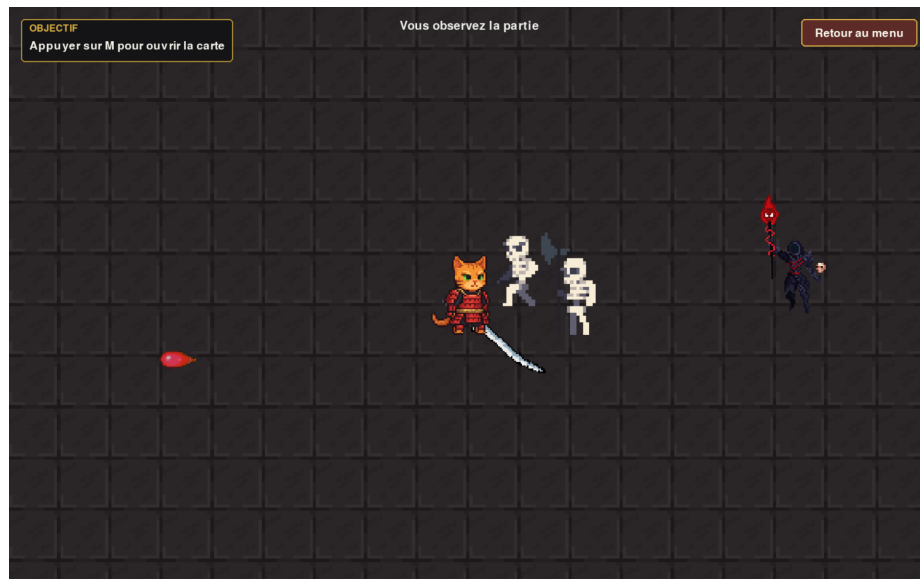


FIGURE 16 – Partie en coopération avec un joueur observateur

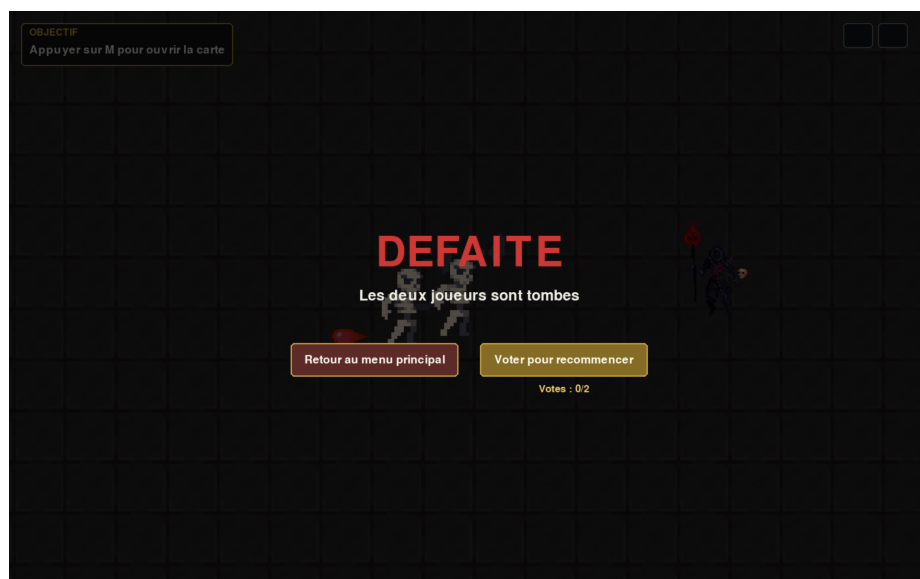


FIGURE 17 – Défaite à deux joueurs et vote de reprise

9 Difficultés rencontrées et choix de conception

9.1 Faire évoluer une carte sans réécrire le jeu

Le placement en dur des monstres était rapide pour un prototype, mais devenait un frein dès que la carte changeait. L'utilisation des objets *Tiled* a demandé de définir des conventions de nommage et de gérer plusieurs types de monstres, mais elle permet aujourd'hui d'ajouter une rencontre sans toucher au code. De même, les collisions par calque ont transformé une tâche manuelle et fragile en une opération de conception de niveau.

9.2 Concilier lisibilité et ambition des adversaires majeurs

Les attaques spectaculaires sont intéressantes seulement si le joueur comprend comment y répondre. Le rayon lumineux du golem était ambitieux, mais son rendu et sa collision manquaient de fiabilité. Le passage à une onde de choc nous a appris qu'une mécanique plus maîtrisée est préférable à un effet complexe qui semble injuste. La seconde phase reste marquante grâce au changement visuel, aux statistiques renforcées et à la nouvelle attaque.

9.3 Synchroniser des systèmes devenus interdépendants

L'arrivée du jeu à deux a révélé que de nombreux systèmes étaient liés : vaincre un adversaire majeur produit une clé, deux clés ouvrent une grille, l'ouverture lance la musique et permet d'affronter le golem, puis la mort du golem affiche la victoire. En réseau, toutes ces transitions doivent être partagées dans le même ordre. Nous avons donc déplacé vers l'hôte les décisions importantes et transmis les états nécessaires aux postes connectés.

Les reprises de partie ont accentué cette difficulté. Relancer uniquement les joueurs laisserait une grille déjà ouverte ou un adversaire majeur déjà vaincu. Le redémarrage doit remettre en cohérence tout l'état du donjon. Ce travail est moins visible qu'une image animée ou un menu, mais il conditionne la fiabilité de l'expérience en coopération.

9.4 Intégrer le travail de plusieurs branches

Le développement en parallèle a permis d'avancer rapidement sur le son, les ennemis, l'apprentissage guidé et le réseau. En contrepartie, certaines fusions ont créé des conflits dans le fichier principal, car toutes ces fonctionnalités interviennent dans la boucle du jeu. La résolution ne consistait pas uniquement à choisir une version : il fallait recomposer les comportements pour conserver à la fois les musiques, les objectifs, les boutons de reprise et les nouvelles règles du jeu à deux.

Cette expérience justifie notre effort de séparation progressive des responsabilités : cartes et collisions dans la gestion de niveau, ennemis dans leurs classes, menu et son dans l'interface, synchronisation dans le serveur. Plus chaque fonction possède un emplacement clair, plus une future évolution est vérifiable et intégrable.

10 Communication et organisation

10.1 Outils de communication

Le serveur *Discord* centralise les échanges techniques, les ressources et les réunions vocales. Il permet de signaler rapidement une anomalie, de montrer une capture ou de demander une vérification après une intégration. *Instagram* est utilisé de manière complémentaire pour des messages rapides et les imprévus.

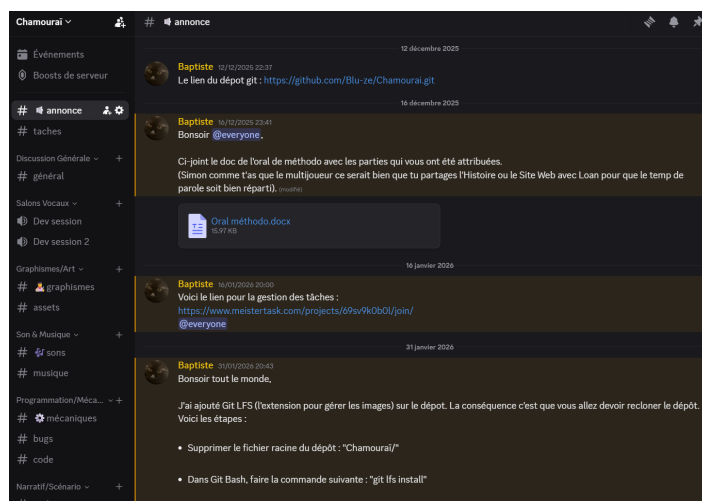


FIGURE 18 – Serveur *Discord* du groupe *Les Chatpitres*

10.2 Gestion des versions

Nous utilisons *Git* pour répartir le développement et conserver l'historique du projet. Afin de réduire les conflits, nous communiquons sur *Discord* avant d'intervenir sur une même branche ou sur un même fichier important. Les images et les sons étant volumineux, nous utilisons également *Git LFS*, une extension de *Git* adaptée au suivi de ces fichiers.

Malgré cette organisation, les fusions de branches ont parfois été délicates. Le fichier principal rassemble plusieurs comportements du jeu, comme les sons, les objectifs ou le jeu à deux ; deux ajouts indépendants peuvent donc modifier les mêmes passages. Certains conflits ont demandé de recomposer manuellement les modifications, avec le risque de perdre une fonction déjà réalisée. Cette difficulté nous a appris à mieux communiquer avant les fusions et à vérifier systématiquement les fonctions importantes après leur résolution.

10.3 Suivi des tâches

Nous organisons le travail à l'aide de *MeisterTask*. Chaque tâche possède un responsable et un état d'avancement, ce qui permet d'identifier les fonctions à intégrer et celles à vérifier. Les réunions hebdomadaires du jeudi matin servent à faire le point, répartir les priorités et résoudre collectivement les blocages les plus importants.

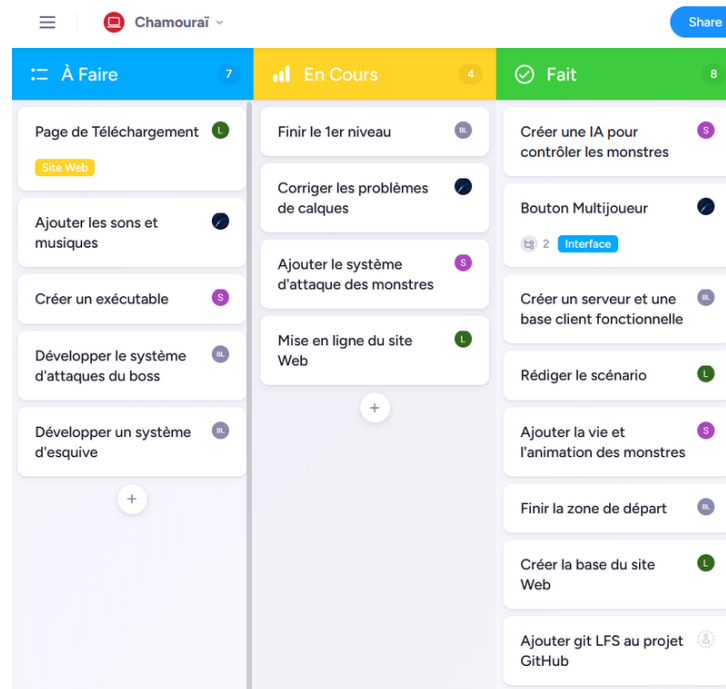


FIGURE 19 – Tableau de suivi des tâches sur *MeisterTask*

Dans les dernières phases, l'organisation des vérifications est devenue aussi importante que l'ajout de fonctionnalités : une mécanique correcte à un joueur doit être revue à deux, après une mort, après une reprise et après une transition de carte.

11 Bilan et perspectives

La version actuelle de *Chamourai* remplit le coeur de notre ambition : le joueur apprend les contrôles dans une introduction guidée, explore un donjon peuplé d'ennemis variés, récupère les clés auprès d'adversaires intermédiaires et termine sur un combat en deux phases contre le golem. Le jeu est jouable seul ou à deux, dispose d'une identité sonore et visuelle, et traite les principaux cas de victoire, de défaite et de reprise.

Les évolutions les plus importantes ne sont pas seulement quantitatives. Le passage à des cartes pilotant les apparitions et collisions facilite la création de contenu. La synchronisation entre les deux joueurs permet à la progression de rester cohérente. Enfin, les choix effectués pour le golem et l'apprentissage guidé montrent notre priorité : proposer une expérience claire, équilibrée et agréable avant de multiplier les effets difficiles à maîtriser.

Pour une version future, nous pourrions compléter les vérifications du réseau sur des connexions moins stables et approfondir le combat contre le golem. Nous avons notamment imaginé son attaque de rayon lumineux avant de l'abandonner, car les ressources graphiques étaient difficiles à exploiter correctement avec *Pygame* ; l'onde de choc en arc de cercle constitue la solution que nous avons su rendre fonctionnelle.

Nous aurions aussi aimé proposer plusieurs niveaux et permettre au joueur de débloquent de nouvelles armes au fil de sa progression. Nous avons finalement préféré concentrer nos efforts sur un unique niveau détaillé, avec une progression complète, plusieurs adversaires et un combat final, plutôt que de multiplier des zones moins abouties.

12 Annexes

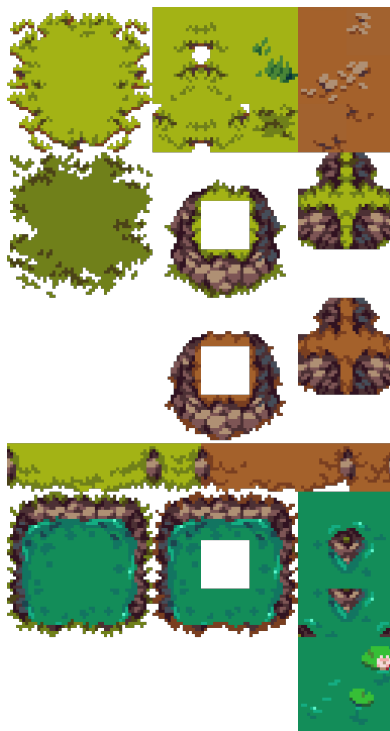


FIGURE 20 – Exemple de feuille de tuiles utilisée pour construire les cartes

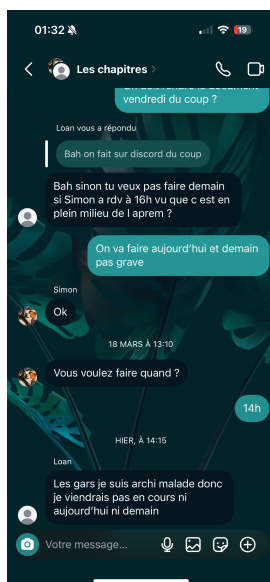


FIGURE 21 – Groupe *Instagram* de communication rapide

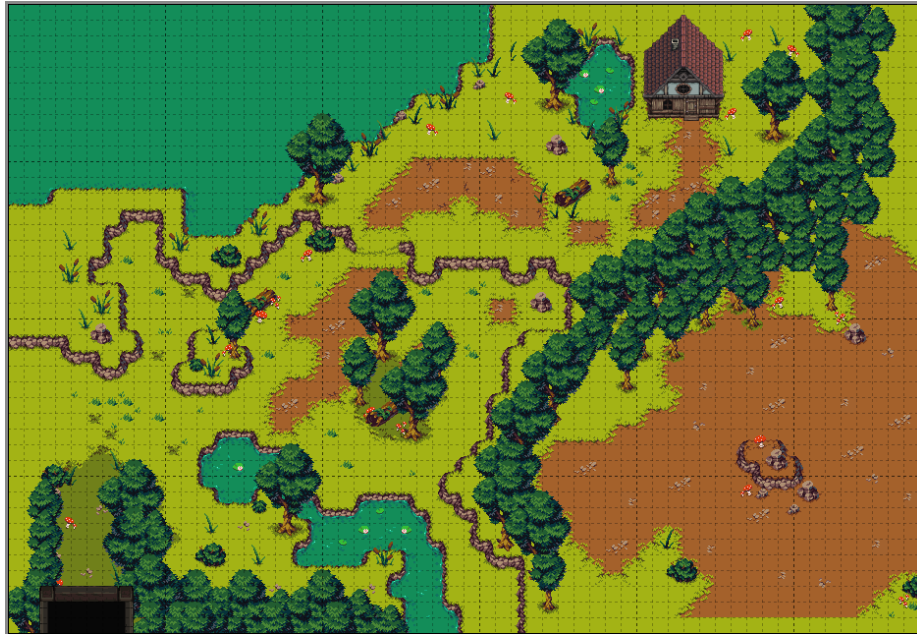


FIGURE 22 – La carte de départ dans son entièreté



FIGURE 23 – L'adversaire intermédiaire nécromancien